



《AI大模型Ragent项目》——检索结果、工具数据、对话历史——最终的Prompt怎么拼

来自： 拿个offer-开源&项目实战



马丁

2026年05月14日 21:58

开篇引言

上一篇把 MCP 参数提取器的内部实现拆干净了，MCP 工具调用子系列到此收尾。回头看一下，到第 13 篇为止，前面所有阶段的数据都已经备齐：

- 第 2、3 篇产出了**会话记忆**——对话摘要加上最近几轮的 history
- 第 4 篇产出了**改写后的子问题**——指代消解、上下文补全后的检索友好查询
- 第 5~9 篇产出了**意图识别结果**——命中了哪些 KB 节点、哪些 MCP 节点，每个节点上挂着什么 promptTemplate 和 promptSnippet
- 第 10、11 篇产出了**5 条精排后的 KB 检索结果**——三个通道 30 条粗排，后处理流水线压到 5 条
- 第 12、13 篇产出了**MCP 工具执行结果**——一份结构化的销售报告、一条用户年假记录，或者别的什么业务数据

这些数据现在分别在 RetrievalContext 的 kbContext、mcpContext、intentChunks 几个字段里，加上 StreamChatContext 里的 history 和 rewriteResult。但大模型只认一样东西——一个 messages 数组。怎么把这堆零件拼装成大模型能消化的形态，就是本篇要讲的事。

为什么要分场景拼 Prompt

1. 三种原料，三种吃法

直觉上，写一套万能模板似乎就够了——不管什么情况，把检索结果和工具数据一股脑塞进去。但实际跑起来会出问题。KB 检索结果是文档片段，模型需要被告知严格基于文档回答，不能编造，链接和图片要保持原格式。MCP 工具数据是结构化 JSON，模型需要被告知把字段名转成业务术语，隐私字段要脱敏，空数据要有合理的兜底回复。如果只有 KB 结果，塞一堆 MCP 相关的规则（JSON 转述、隐私脱敏）就是噪音；反过来，只有 MCP 数据时，塞 KB 的块级引用约束也毫无意义。所以 Ragent 把 Prompt 组装分成了三个场景，每个场景用不同的 System Prompt 模板：

场景	判定条件	System Prompt 模板	侧重点
KB_ONLY	只有 KB 检索结果	answer-chat-kb.st	文档问答：块级引用、禁止编造、链接图片处理
MCP_ONLY	只有 MCP 工具数据	answer-chat-mcp.st	数据转述：JSON → 自然语言、隐私脱敏、异常处理
MIXED	KB 和 MCP 都有	answer-chat-mcp-kb-mixed.st	综合回答：实时数据准确性优先于文档、资讯等

还有一个特殊情况——所有意图都命中了 SYSTEM 类型节点（打招呼、自我介绍等），这在第 1 篇讲流水线时提过，handleSystemOnly() 会提前短路，压根不走 Prompt 组装这条路。

2. 场景判定的代码

场景判定的逻辑在 RAGPromptService.plan() 方法里，简洁到只有四个 if：

```
private PromptBuildPlan plan(PromptContext context) {
    if (context.hasMcp() && !context.hasKb()) {
        return planMcpOnly(context);    // 只有 MCP
    }
    if (!context.hasMcp() && context.hasKb()) {
        return planKbOnly(context);    // 只有 KB
    }
    if (context.hasMcp() && context.hasKb()) {
        return planMixed(context);    // 两者都有
    }
    throw new IllegalStateException(...); // 不可能走到这里
}
```

判定依据就是 PromptContext 上的两个方法——hasMcp() 和 hasKb()，各自检查对应的上下文字符串是否非空。到这一步时，检索引擎已经执行完了，结果有没有就是一个字符串是否为空的事。

为什么不会出现两者都为空的情况？因为流水线在 `retrieve()` 之后有一个 `handleEmptyRetrieval()` 短路点——如果检索结果全空，直接回复未检索到与问题相关的文档内容就结束了，不会走到 `Prompt` 组装。

消息数组的骨架

1. 四段式结构

不管哪个场景，最终的 `messages` 数组都遵循同一个骨架：

```
[system]    系统提示词（角色定义 + 回答规则）
[...history...] 对话历史（摘要 + 近期多轮对话）
[user]      证据 + 问题（KB 文档 / MCP 数据 + 用户问题，合并为一条 user 消息）
```

`RAGPromptService.buildStructuredMessages()` 就是按这个顺序组装的：

```
public List<ChatMessage> buildStructuredMessages(PromptContext context,
                                                  List<ChatMessage> history,
                                                  String question,
                                                  List<String> subQuestions) {

    List<ChatMessage> messages = new ArrayList<>();

    // 1. 系统提示词
    String systemPrompt = buildSystemPrompt(context);
    if (StrUtil.isNotBlank(systemPrompt)) {
        messages.add(ChatMessage.system(systemPrompt));
    }

    // 2. 对话历史（含摘要）
    if (CollUtil.isNotEmpty(history)) {
        messages.addAll(history);
    }

    // 3. 证据 + 问题（合并为一条 user message）
    String evidenceBody = buildEvidenceBody(context);
    String userQuestion = buildUserQuestion(question, subQuestions);
    String userContent = mergeEvidenceAndQuestion(evidenceBody, userQuestion);
    if (StrUtil.isNotBlank(userContent)) {
        messages.add(ChatMessage.user(userContent));
    }

    return messages;
}
```

四行核心逻辑，对应四段内容。接下来逐段拆解。

2. 第一段：系统提示词

系统提示词决定了模型以什么角色、遵循什么规则来回答。`buildSystemPrompt()` 的逻辑分两步：

1. 调用 `plan()` 判定场景，得到 `PromptBuildPlan`
2. 优先使用 `Plan` 里的 `baseTemplate`（来自意图节点的 `promptTemplate`），没有就用场景对应的默认模板

```
public String buildSystemPrompt(PromptContext context) {
    PromptBuildPlan plan = plan(context);
    String template = StrUtil.isNotBlank(plan.getBaseTemplate())
        ? plan.getBaseTemplate()
        : defaultTemplate(plan.getScene());
    return StrUtil.isBlank(template) ? "" : PromptTemplateUtils.cleanupPrompt(template);
}
```

defaultTemplate() 根据场景返回对应的模板文件：

```
private String defaultTemplate(PromptScene scene) {
    return switch (scene) {
        case KB_ONLY -> templateLoader.load(RAG_ENTERPRISE_PROMPT_PATH);
        case MCP_ONLY -> templateLoader.load(MCP_ONLY_PROMPT_PATH);
        case MIXED    -> templateLoader.load(MCP_KB_MIXED_PROMPT_PATH);
        case EMPTY    -> "";
    };
}
```

三个场景对应三个模板文件，各自有不同的规则侧重。拿 answer-chat-kb.st (KB_ONLY 场景) 来说，它定义了：

- 角色：**熟悉公司内部助手
- 信息源约束：**<documents> 标签内的文字是唯一信息源，没出现的内容不能编造
- 块级引用规则：**每个子问题只用对应的 <document> 块内容，禁止跨块引用
- 链接和图片处理：**链接必须保留完整 URL，图片保持 ![描述](URL) 格式
- 格式规范：**简单问题简单答，多子问题用二级标题区分

而 answer-chat-mcp.st (MCP_ONLY 场景) 则完全换了一套规则：

- 角色：**企业智能数据助手
- 信息源约束：**仅基于 <tool-data> 标签内的数据回答
- 数据格式化：**3 条以上用表格，1~2 条用分点，单一结论直接一句话
- 字段名转述：**create_time → 创建时间，去技术化
- 隐私合规：**默认不输出手机号、身份证号、邮箱、薪酬等敏感信息

answer-chat-mcp-kb-mixed.st (MIXED 场景) 则兼顾两者，额外增加了**冲突处理规则**——当数据和文档对同一事实有不一致时，以数据为准，文档只补充背景和定义。

3. 第二段：对话历史

对话历史直接从 StreamChatContext.history 里取，原封不动地插入 messages 数组。这个 history 是第 2、3 篇讲过的会话记忆的产物——早期对话被压缩成摘要（作为 system 类型消息），最近几轮保留原始的 user/assistant 交替。
把 history 放在系统提示词之后、证据之前，是一个刻意的位置选择。系统提示词定义了规则，history 提供了对话上下文，证据紧贴用户问题——这个顺序让模型在看到证据和问题时，已经建立了角色认知和对话上下文，能更好地理解用户的真实意图。

4. 第三段：证据体

证据体是 KB 检索结果和 MCP 工具数据的合并产物。buildEvidenceBody() 把两种上下文分别用不同的 XML 标签包裹，然后拼在一起：

```
private String buildEvidenceBody(PromptContext context) {
    StringBuilder sb = new StringBuilder();
    if (StrUtil.isNotBlank(context.getMcpContext())) {
        sb.append(renderSection("mcp-evidence",
                                Map.of("body", context.getMcpContext().trim())));
    }
    if (StrUtil.isNotBlank(context.getKbContext())) {
        if (!sb.isEmpty()) {
            sb.append("\n\n");
        }
        sb.append(renderSection("kb-evidence",
                                Map.of("body", context.getKbContext().trim())));
    }
    return sb.toString().trim();
}
```

mcp-evidence 和 kb-evidence 是 context-format.st 模板文件里的两个 section，渲染后分别生成 <tool-data>...</tool-data> 和 <documents>...</documents> 标签。模型在 System Prompt 里被告知从这两个标签里取数据，标签名和 System Prompt 中的引用完全对应。

5. 第四段：用户问题

用户问题的拼装有两种情况：

```
private String buildUserQuestion(String question, List<String> subQuestions) {
    if (CollUtil.isEmpty(subQuestions) && subQuestions.size() > 1) {
        // 多个子问题：编号列表
        String numbered = IntStream.range(0, subQuestions.size())
            .mapToObj(i -> (i + 1) + ". " + subQuestions.get(i))
            .collect(Collectors.joining("\n"));
        return renderSection("multi-questions", Map.of("questions", numbered));
    }
    // 单个问题
    return renderSection("single-question", Map.of("question", question));
}
```

单个问题包在 <question> 标签里，多个子问题包在 <questions> 标签里并带编号。模板定义在 context-format.st：

```
--- section: single-question ---
<question>{question}</question>

--- section: multi-questions ---
<questions>
{questions}
</questions>
```

证据体和用户问题最终通过 mergeEvidenceAndQuestion() 合并为一条 user 消息，中间用双换行分隔。

意图节点的 Prompt 注入

1. promptTemplate 和 promptSnippet 的区别

第 5 篇讲意图图时提到过，每个 IntentNode 上可以挂两个可选的 Prompt 配置：

promptTemplate：完整的 System Prompt 模板，直接替换场景默认模板

promptSnippet：短规则片段，注入到上下文的 <rules> 标签中

两者的用途完全不同。promptTemplate 是大锤子——某个业务场景有一套独特的回答规范（比如法务合规场景要求每句话都标注法条来源），直接整体替换 System Prompt。promptSnippet 是小钉子——在默认模板的基础上追加几条针对性的规则（比如退货政策节点要求“价格数据保留两位小数”）。

2. promptTemplate 的生效规则

promptTemplate 只在单意图命中时才有机会生效。看 planPrompt() 方法：

```
private PromptPlan planPrompt(List<NodeScore> intents,
                               Map<String, List<RetrievedChunk>> intentChunks) {
    // 先剔除未命中检索的意图
    List<NodeScore> retained = safeIntents.stream()
        .filter(ns -> {
            String key = nodeKey(ns.getNode());
            List<RetrievedChunk> chunks = intentChunks.get(key);
            return CollUtil.isEmpty(chunks);
        })
        .toList();

    if (retained.size() == 1) {
        IntentNode only = retained.get(0).getNode();
        String tpl = StrUtil.emptyOrNull(only.getPromptTemplate()).trim();
        if (StrUtil.isNotBlank(tpl)) {
            // 单意图 + 有模板 → 用节点模板
            return new PromptPlan(retained, tpl);
        }
    }
}
```

```
    }  
    // 单意图 + 无模板 → 走默认模板  
    return new PromptPlan(retained, null);  
}  
// 多意图 → 统一默认模板  
return new PromptPlan(retained, null);  
}
```

为什么多意图不能用 `promptTemplate`? 因为如果两个意图节点各自定义了不同的 `promptTemplate`, 合并起来可能互相矛盾。与其搞复杂的合并逻辑, 不如统一走默认模板, 让 `promptSnippet` 来处理节点级的规则追加。

3. `promptSnippet` 的注入和合并

`promptSnippet` 在 `DefaultContextFormatter` 里注入, 具体方式取决于单意图还是多意图。

单意图——`snippet` 直接放进 `<rules>` 标签:

```
private String formatSingleIntentContext(NodeScore nodeScore,  
    Map<String, List<RetrievedChunk>> rerankedByIntent, int topK) {  
    String snippet = StrUtil.emptyIfNull(nodeScore.getNode().getPromptSnippet()).trim();  
    String body = joinChunkTexts(chunks, topK);  
    return renderKbSection(renderSnippetRules(snippet), body);  
}
```

多意图——所有节点的 `snippet` 去重后编号合并:

```
private String formatMultiIntentContext(List<NodeScore> kbIntents,  
    Map<String, List<RetrievedChunk>> rerankedByIntent, int topK) {  
    List<String> snippets = kbIntents.stream()  
        .map(ns -> ns.getNode().getPromptSnippet())  
        .filter(StrUtil::isNotBlank)  
        .map(String::trim)  
        .distinct()  
        .toList();  
  
    String snippetSection = "";  
    if (!snippets.isEmpty()) {  
        String numberedRules = IntStream.range(0, snippets.size())  
            .mapToObj(i -> (i + 1) + ". " + snippets.get(i))  
            .collect(Collectors.joining("\n"));  
        snippetSection = renderSnippetRules(numberedRules);  
    }  
    // ...  
}
```

假设用户问了一个跨部门的问题, 同时命中了退货政策和售后服务两个意图节点, 退货政策节点的 `snippet` 是价格数据保留两位小数, 售后服务节点的 `snippet` 是时间节点用年月日格式。合并后会变成:

```
<rules>  
1. 价格数据保留两位小数  
2. 时间节点用年月日格式  
</rules>
```

渲染出来的 KB 上下文结构是 `<rules> + <content>`, 其中 `rules` 是意图节点追加的约束, `content` 是检索到的文档片段。MCP 上下文也有类似的结构——`<rules> + <data>`。

KB 上下文为什么放 `user` 而不放 `system`

这是 Prompt 组装中一个值得单独拿出来讲的设计决策。

1. 直觉上的选择

很多 RAG 系统会把检索到的文档放进 System Prompt——既然 System 里定义了基于以下知识回答，把知识也放在 System 里，逻辑上顺理成章。Ragent 早期也是这么做的。

2. 为什么会出问题

在多轮对话场景下，System Prompt 中嵌入的 KB 文档其有效性会随对话轮次增加而下降。这与长上下文 LLM 中的 **Lost in the Middle 现象** (Liu et al., 2023) 有关：模型对上下文头部和尾部 token 的利用率高于中部。当 KB 文档被包裹在大段角色定义和规则之中，又被多轮历史对话推离了“尾部”，它实际上落在了注意力的低谷区。此外，后续对话内容也会对 System Prompt 的指令产生稀释和干扰。

这个效应在历史对话较长、或使用上下文窗口较小的模型时会更明显。

当对话进行了十几轮之后，messages 数组大致长这样：

```
[system]      系统提示词 + KB 文档（如果放在这里）
[system]      对话摘要
[user]        第 8 轮问题
[assistant]   第 8 轮回答
[user]        第 9 轮问题
[assistant]   第 9 轮回答
[user]        第 10 轮问题（当前）
```

KB 文档在 messages[0]，当前问题在 messages[最后]，中间隔了几十条历史消息。一个典型的 bad case 是：用户在第 9 轮问“刚才那个参数的默认值是多少”，KB 文档里明确写了，但模型却回答“根据上下文我没看到相关信息”——文档就在 System Prompt 里，只是被规则和历史对话夹在了中间，模型的注意力没分配过去。

3. 放到 user 里的三个好处

第一，距离问题更近。 把证据放在最后一条 user 消息中（与当前问题合并），模型的注意力天然集中在数组末尾。证据和问题紧密相邻，模型更容易把两者关联起来。

第二，System Prompt 保持精简，且对缓存友好。 System Prompt 只放角色定义和回答规则，不掺杂可能几千 Token 的动态文档内容。一方面规则的执行力不会被大段文档稀释；另一方面，主流 API（如 Claude 的 prompt caching、OpenAI 的 system message 处理）通常会对稳定的 System Prompt 做缓存优化，把每轮都在变化的 KB 文档塞进去会破坏缓存命中率，显著增加成本和延迟。

第三，贴合主流 RAG 微调数据的格式。 大多数开源和闭源模型的 RAG 相关 SFT 数据，都是把检索文档放在 user 消息中（紧邻问题）的格式。把证据放在 user 里，相当于让推理时的输入分布与训练时一致，模型更容易触发引用文档作答的行为模式，而不是把检索结果当成背景知识一笔带过。

调整后的 messages 数组变成这样：

```
[system]      系统提示词（只有规则，没有文档）
[system]      对话摘要
[user]        第 8 轮问题
[assistant]   第 8 轮回答
[user]        第 9 轮问题
[assistant]   第 9 轮回答
[user]        证据（KB 文档 + MCP 数据）+ 当前问题
```

证据和问题在同一条 user 消息里，紧贴数组末尾——模型注意力最集中的位置。

你可能会问：对话摘要不也在中间吗？是的，但摘要的作用是兜底的长程记忆，本来就不要求逐字精确召回；而 KB 文档是当前回答的核心证据，必须放在注意力高地上。两者的优先级不同，位置策略也不同。

Temperature 分档

1. 三档温度

StreamChatPipeline 在调用大模型时，根据场景设定不同的 Temperature 和 Top-P：

```
// RAG 场景
ChatRequest chatRequest = ChatRequest.builder()
    .messages(messages)
    .thinking(deepThinking)
```

```
.temperature(ctx.hasMcp() ? 0.3D : 0D)
.topP(ctx.hasMcp() ? 0.8D : 1D)
.build();

// SYSTEM 场景（打招呼、闲聊）
ChatRequest req = ChatRequest.builder()
    .messages(messages)
    .temperature(0.7D)
    .thinking(false)
    .build();
```

汇总起来是三档：

场景	Temperature	Top-P	设计理由
KB_ONLY	0.0	1.0	严格依据文档，零随机性，答案必须从文档里来
MCP_ONLY / MIXED	0.3	0.8	工具数据需要合理串联和解读，允许轻度发散
SYSTEM（闲聊）	0.7	默认	开放对话，允许创造性表达

2. 每档的工程理由

KB_ONLY 用 0.0，是因为知识库问答的核心要求是忠实度。用户问“年假有几天”，文档里写着 15 天就应该答 15 天，不能因为模型的随机性变成 14 天或 16 天。Temperature 为 0 意味着每次都选概率最高的 Token，完全消除随机性。配合 topP=1.0（不做概率截断），让模型在全词表上选最优解。

MCP 场景用 0.3，是因为工具返回的数据往往是原始 JSON，需要模型做一定程度的加工——把字段名翻译成业务术语、把多条记录组织成表格、在数据之间建立关联（本月销售额环比上月增长了 15%）。这些加工需要一定的灵活度，但又不能太大。0.3 是一个经验值——足够让模型组织出自然流畅的回答，又不会偏离原始数据。topP=0.8 配合使用，把概率最低的 20% 长尾词汇剪掉，减少生成罕见表达的可能。

SYSTEM 场景用 0.7，是因为闲聊和打招呼不需要严格依据任何资料。“你好，有什么可以帮到你？”这类回答越自然越好，Temperature 高一些让每次打招呼的措辞不完全一样，避免机械感。

完整示例：一个 MIXED 场景的拼装过程

用一个具体请求走一遍完整的拼装流程。假设用户在第 5 轮对话中问了一个跨 KB 和 MCP 的问题：

用户问题：“华东区的退货政策是什么？顺便查一下华东区本月的退货数据。”

第 4 篇改写后的子问题：

- 1. 华东区的退货政策是什么
- 2. 华东区本月的退货数据

第 5~9 篇意图识别结果：

- 子问题 1 命中 KB 节点退货政策（kind=KB）
- 子问题 2 命中 MCP 节点退货查询（kind=MCP）

第 10~11 篇 KB 检索结果（子问题 1）：

```
退货政策文档片段：7 天无理由退货，商品需保持原包装.....
```

第 12~13 篇 MCP 工具执行结果（子问题 2）：

```
{"region": "华东", "period": "本月", "totalReturns": 42, "totalAmount": 128500}
```

场景判定：hasMcp()=true, hasKb()=true → MIXED

最终 messages 数组：

```
[
  {
    "role": "system",
    "content": "# 角色\n\n你是专业、稳重且友好的企业智能助手，能够综合业务数据与知识文档，给出清晰、实用的回答。\\n",
  },
  {
    "role": "system",
    "content": "<conversation-summary>\n用户之前咨询了产品版本区别和定价信息.....\n</conversation-summary>"
  },
  {
    "role": "user",
    "content": "华东区有哪些产品版本？"
  },
  {
    "role": "assistant",
    "content": "华东区目前销售企业版、专业版和基础版三个版本....."
  },
  {
    "role": "user",
    "content": "<tool-data>\n{\"region\": \"华东\", \"period\": \"本月\", \"totalReturns\": 42, \"t"
  }
]
```

看看这个数组的结构：

- 1. messages[0]：MIXED 场景的 System Prompt，定义了综合回答的角色和规则
- 2. messages[1]：对话摘要，让模型知道之前聊过产品版本和定价
- 3. messages[2~3]：最近一轮对话历史
- 4. messages[4]：证据 + 问题——MCP 数据在 <tool-data> 标签里，KB 文档在 <documents> 标签里（带 <rules> 规则片段），两个子问题在 <questions> 标签里带编号

每一段从哪个阶段来、为什么放在这个位置，在前文都已经讲清楚了。

调用参数：因为是 MIXED 场景（hasMcp()=true），所以 temperature=0.3，topP=0.8。

核心类速查表

类名	职责	关键方法
RAGPromptService	Prompt 组装核心入口	buildStructuredMessages(), buildSystemPrompt(), plan()
PromptContext	封装一次 Prompt 组装的全部输入	hasMcp(), hasKb()
PromptBuildPlan	场景规划结果	持有 PromptScene、baseTemplate
PromptPlan	意图级模板选择结果	持有 retainedIntents、baseTemplate
PromptScene	场景枚举	KB_ONLY、MCP_ONLY、MIXED、EMPTY
DefaultContextFormatter	KB / MCP 上下文格式化	formatKbContext(), formatMcpContext()
PromptTemplateLoader	模板加载与渲染	load(), renderSection()
StreamChatPipeline	流水线编排	streamLLMResponse(), streamSystemResponse()

相关模板文件：

文件	用途
prompt/answer-chat-kb.st	KB_ONLY 场景 System Prompt
prompt/answer-chat-mcp.st	MCP_ONLY 场景 System Prompt

prompt/answer-chat-mcp-kb-mixed.st	MIXED 场景 System Prompt
prompt/answer-chat-system.st	SYSTEM 场景 System Prompt
prompt/context-format.st	上下文格式化模板（11 个 section）

小结与下一篇预告

本篇拆解了 Prompt 组装的完整过程，核心要点：

- 1. 场景分流——根据 hasMcp() 和 hasKb() 判定 KB_ONLY、MCP_ONLY、MIXED 三种场景，每种场景用不同的 System Prompt 模板，规则侧重点各不相同
- 2. 消息数组的四段式骨架——system（角色和规则）→ history（摘要和近期对话）→ evidence + question（证据和问题合并为一条 user 消息），顺序固定
- 3. 意图节点的 Prompt 注入分两层——promptTemplate 是大锤子，单意图时整体替换 System Prompt；promptSnippet 是小钉子，注入到上下文的 <rules> 标签中，多意图时编号合并
- 4. KB 上下文放 user 而不放 system——避免 Lost in the Middle，让证据紧贴问题放在数组末尾，模型注意力最集中的位置
- 5. Temperature 三档——KB_ONLY 用 0.0（忠实文档），MCP 场景用 0.3（合理串联数据），SYSTEM 闲聊用 0.7（自然表达）
- 6. 证据体用 XML 标签区分类型——<documents> 包裹 KB 文档，<tool-data> 包裹 MCP 数据，标签名与 System Prompt 中的引用一一对应

Prompt 拼装完成，ChatRequest 也构建好了——messages 数组、temperature、topP、是否开启深度思考，所有调用参数都就位了。下一步，就是把这个 ChatRequest 丢给大模型，让答案一个字一个字蹦出来。流式生成怎么控制、SSE 推送怎么封装、客户端断连怎么处理——接下来开始讲。

